검증 기반 코드 재생성 파이프라인

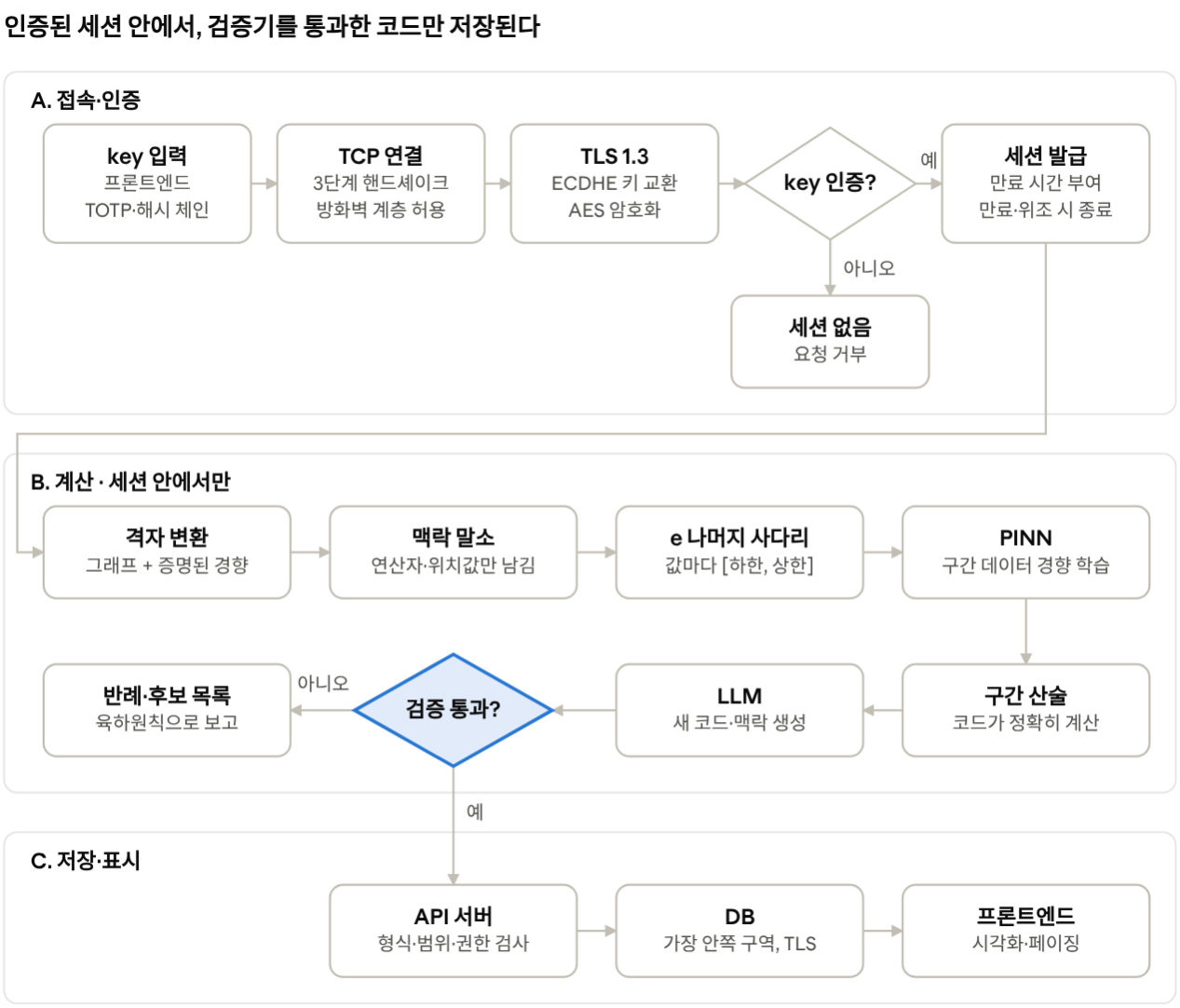
Nov 2, 2026 · @an sung ju

목적

이 파이프라인은 코드의 맥락을 지운 뒤 연산자와 위치값만으로 계산한다. 그다음 LLM이 새 코드를 쓰고, 검증기를 통과한 코드만 저장해 보여 준다. 성격은 연구개발이 아니라 버그 판정과 검증(QA) 쪽이다.

원칙은 하나다. 근사는 PINN과 LLM이 하고, 판정은 검증기만 한다.

전체 흐름



파이프라인 흐름 · 인증 → 계산 → 저장, 판정 두 곳

key 인증에 실패하면 세션이 생기지 않는다. 검증에 실패한 코드는 실행되지 않고 반례나 후보 목록으로 보고된다.

A. 접속·인증·세션

인증에 성공해야 세션이 생기고, 계산은 그 세션 안에서만 돈다.

1. **key 입력:** 사용자가 프론트엔드에서 key를 입력한다.
2. **TCP 연결:** 클라이언트와 서버가 3단계 핸드셰이크(SYN → SYN-ACK → ACK)로 TCP 연결을 연다. 순서와 도착을 보장해야 하므로 UDP는 쓰지 않는다. UDP가 꼭 필요하면 순서와 암호화를 보장하는 QUIC을 쓴다.
3. **방화벽 계층:** 이 핸드셰이크는 바깥 → DMZ → 내부 구역의 방화벽과 라우터 접근 제어 목록(ACL)이 모두 허용해야 지나간다. 보안을 정하는 것은 방화벽 개수가 아니라 층마다 다른 규칙이다.
4. **TLS 1.3:** TCP 위에서 TLS 핸드셰이크를 한다. 키 교환은 ECDHE(또는 RSA)가, 데이터 암호화는 AES가 맡는다. 서버끼리는 상호 인증서(mTLS)로 서로를 확인한다.
5. **인증:** 서버가 key를 확인한다. 방식은 TOTP(시간 기반 일회용 비밀번호)나 램포트 해시 체인이다. 해시 체인은 비밀값을 n번 해시해 두고, 로그인마다 한 단계 이전 값을 받는다. 점화식을 거꾸로 밟는 인증이다.
6. **세션 발급:** 인증에 성공하면 세션(세션 ID, 만료 시간)을 발급한다. 실패하면 세션을 만들지 않고 요청을 거부한다.
7. **세션 유지와 종료:** 이후 모든 요청은 세션을 확인하고, HMAC 서명과 일회용 난수(nonce)로 재전송을 막는다. 세션이 만료되거나 위조되거나 key가 무효가 되면 세션을 종료한다.

B. 계산

코드는 격자와 연산자로 바뀌고, 격자에서 증명된 경향이 먼저 나온다. 사다리 구간과 PINN 경향을 거쳐 LLM이 새 코드를 쓰고, 검증기를 통과한 코드만 실행된다.

1. **격자 변환:** 코드를 격자, 곧 가중 그래프(마디 무게 V, 변 무게 T)로 바꾼다. 조건문은 참·거짓 갈래를 하나씩 여는 결정 열거로 다룬다. 반복문은 반복 횟수 대표값 몇 개로 모든 n을 보인다. 시각화는 이 격자를 그려 주는 선택 기능이다.
2. **격자 경향:** 격자 구조 (V, T)만으로 해가 시간에 따라 어떻게 움직일지가 증명으로 나온다. 근사가 아니라 증명이라서 판정에 쓸 수 있다. 선형 확산형 연산자에서 행합이 0이고, V·L이 대칭이며, 비대각이 음이 아니면 다음이 성립한다.
   - **총량 보존:** 영 유량 경계에서 $\Sigma\, V_i\, u_i$가 변하지 않는다. 문서의 상수 보존(R2)·질량 보존(R2′) 규칙이다.
   - **에너지 감소:** $\Sigma\, V_i\, u_i^2$가 늘지 않는다.
   - **최대·최소 원리:** 최댓값은 커지지 않고 최솟값은 작아지지 않는다. 문서 8절 이산 비교 원리가 쓰는 메츠러 성질이다.
   - **시간 걸음 한계:** 명시적 오일러는 $\tau \leqslant \min_i V_i / \Sigma_j T_{ij}$일 때 위 세 성질을 지키고, 암시적 오일러는 모든 τ에서 지킨다.
   - **변화가 몰리는 곳:** 변마다 흐름 $T_{ij}\,(u_j - u_i)$를 재면 변화가 큰 자리가 보인다.

3. **맥락 말소:** 변수명, 주석, 로직 같은 맥락을 지우고 연산자 그래프와 위치값만 남긴다. 업계에서는 이를 중간 표현(IR)이라고 부른다. 차이식이 0이나 상수로 접히는 항과 갈래는 지우되, return 0이 아니라 "제거됨" 표시로 돌려준다. 실제 계산 결과 0과 섞이지 않게 하기 위해서다.
4. **e 나머지 사다리:** 위치마다 값에 구간 [하한, 상한]을 씌운다. e는 부분합 S_n과 꼬리 1/(n!·n)으로 감싸고, 반올림 오차는 반올림 부등식으로 따로 잡는다.
5. **PINN:** 구간 데이터와 방정식을 받아 위치·시간에 따른 경향을 학습한다. 예측이 구간 밖으로 나간 만큼의 벌점과 방정식 잔차의 벌점을 더해 손실로 쓴다. 결과는 "경향(미검증)"으로 표시해 넘긴다. PINN 경향이 격자 경향과 어긋나면(예: 총량이 늘어난다고 예측) 그 경향은 버린다. 사다리 자체의 경향은 1/(n!·n)으로 이미 알려져 있어 배울 대상이 아니다.
6. **구간 산술:** 구간 상수끼리의 사칙연산은 코드가 유리수·구간 산술로 정확하게 한다. PINN과 LLM은 산술을 하지 않는다.
7. **LLM:** 연산자 그래프, 구간 상수, 경향을 받아 새 코드와 새 맥락을 생성한다. 요청마다 기억 없이(무상태) 처리하고, DB 자격증명을 갖지 않는다.
8. **검증기:** 새 코드가 원래 연산자 그래프와 구간을 재현하는지 판정한다. 판정은 증명, 반례, 좁혀 가는 후보 목록 중 하나이고, 육하원칙으로 보고한다. 통과한 코드만 실행한다. 경로가 폭발하면 예산에서 멈추고, 확인한 부분은 확정하고, 남은 부분은 후보 목록으로 넘긴다.

C. 저장·표시

DB에는 API 서버가 검사한 값만 들어간다.

1. **API 서버:** 값의 형식·범위·권한을 검사하고, 매개변수화된 쿼리로만 DB에 쓴다. LLM은 이 API만 부를 수 있다.
2. **DB:** 가장 안쪽 구역에 두고, DB 포트는 API 서버에게만 연다. 연결은 TLS로 암호화하고, 역할별로 최소 권한만 준다.
3. **프론트엔드:** 판정 결과와 격자를 차트와 그림으로 보여 준다. 목록은 서버 쪽 페이지네이션(커서 방식)으로 나눠 불러온다.

설계 원칙

근사는 PINN과 LLM이 하고, 판정은 검증기만 한다. 나머지 원칙은 모두 이 분리를 지키기 위한 것이다.

- **근사와 판정의 분리:** PINN의 경향과 LLM의 코드는 힌트다. 실행 여부는 검증기가 정한다.
- **증명된 경향이 근사 경향을 거른다:** 격자에서 증명된 경향(보존, 최대 원리)은 판정에 쓰고, PINN의 경향은 이와 어긋나면 버린다.
- **정확한 산술은 코드가:** 신경망과 LLM은 계산을 근사하므로 산술을 맡기지 않는다.
- **"제거됨"과 0의 구분:** 지운 값을 0으로 돌려주면 식과 값의 어긋남이 생긴다.
- **암호는 직접 만들지 않는다:** 방법이 공개돼도 키만 비밀이면 안전해야 한다(케르크호프스 원칙). 그래서 TLS, TOTP, HMAC 같은 표준을 쓴다.

- **데이터 최소화:** 코드 원문은 밖으로 나가지 않고, 연산자와 위치값만 넘긴다.
- **최소 권한:** LLM은 DB 자격증명이 없고, 모든 요청은 서버가 권한을 검사한다. 프롬프트로는 권한을 넘을 수 없다.
- **다층 방어:** 방화벽을 여러 겹 두되, 층마다 막는 것이 달라야 의미가 있다. 방화벽은 출발지와 포트를 걸러낼 뿐 내용은 보지 않는다.

카톡 설명에서 고친 것

발상은 대부분 그대로 두고, 틀리거나 위험한 표현 13개를 고쳤다.

원래 표현	고친 표현	이유
격자가 조건문·반복문·불린을 시각화해서 빨리 찾음	조건문은 결정 열거, 반복문은 대표 크기로 다루고, 격자는 그래프로 바꾼 계산 구조	찾는 힘은 판정 규칙에서 나오고, 시각화는 선택 기능
계산 비용은 PINN으로	PINN은 사다리 구간 데이터의 경향을 학습	PINN은 근사기라 판정을 못 함
PINN이 산술연산	산술은 코드가 구간 산술로	신경망은 1 + 1도 근사함
점화식 중 계산 말소(return 0)	"제거됨" 표시	실제 계산 결과 0과 구분
공간은 선형, 시간은 비선형	공간 스킴은 그래프 상수, 시간 스킴은 선형이면 오일러 항등식·비선형이면 적분과 부등식	공간에도 비선형(칸–힐리어드)이 있음
시간은 유한요소가 안 됨	시간을 따로 다루는 이유는 인과성(처음 값에서 앞으로만 진행)	시공간 갤러킨 같은 시간 유한요소도 있음
연산자만 남기고 맥락을 지우는 블록체인	맥락 말소(중간 표현), 검증 결과는 육하원칙으로 공개	블록체인은 이 구조가 아님
AES·RSA 기반 시각화 패턴을 3번 꼬아 인증	TLS(AES·RSA는 그 부품) + TOTP 또는 해시 체인	직접 만든 암호는 깨짐
TCP on = 방화벽 3개 이상 통과	TCP는 3단계 핸드셰이크로 열리고, 방화벽 계층이 그 연결을 허용	연결과 허용은 다른 층
TCP·UDP로 연결	TCP 위 TLS(또는 QUIC)	순서 보장이 필요함
LLM이 맥락을 지우고 산술한 뒤 DB에 넘김	LLM은 새 코드만 쓰고, DB에는 API 서버가 검사한 값만	프롬프트 조작은 방화벽을 그대로 통과함
백엔드 세션을 열고 인증 실패 시 종료	인증 성공 후에만 세션 발급	인증 전 계산은 비용·노출 위험

원래 표현	고친 표현	이유
연산자만 남기면 프롬프트가 DB 권한을 못 뚫음	권한 설계(최소 권한, 서버 쪽 검사)로 막음	맥락 말소는 권한을 막지 못함

구현 상태와 한계

검증기 쪽은 이미 만들어 시험했고, 나머지 단계는 설계 단계다. 근거는 문서 「식과 값이 어긋날 때 생기는 모든 에러의 검증과 해결」의 절 번호다.

단계	상태	근거
격자(그래프 V, T), 공간 이산 판별	구현·시험됨	4.4절, 19.1절
결정 열거, 대표 크기, 경로 조건	구현·시험됨	20.3절, 20.4절, 22.10절
e 나머지 사다리, 반올림 부등식	구현·시험됨	5.1절, 11절
검증기 판정(증명·반례·후보 목록, 육하원칙)	구현·시험됨	제5–6부, 정의 25.1–25.3
맥락 말소(중간 표현)	설계	관련 장치: 20.2절(정확 산술로 코드의 식을 읽음)
PINN 경향 학습	설계	문서의 PINN(17절)은 검증 대상이지 경향 학습기가 아님
LLM 코드 생성	설계	제2부 검증 프롬프트가 판정 기준
인증·네트워크·백엔드·프론트	설계	표준 기술(TLS, TOTP, API 서버) 사용

한계는 네 가지다.

- **경로 폭발:** 최악의 경우를 다항 시간에 푸는 방법은 알려져 있지 않다. 예산에서 멈추고, 남은 부분은 후보 목록과 표본 보증(실패율 ε 이하를 신뢰도 1 − δ로)으로 넘긴다.
- **계산할 수 없는 문제:** 정지 문제류는 정의 25.2의 좁혀 가는 후보 목록으로 답한다.
- **근사는 판정이 아니다:** PINN 경향과 LLM 코드는 검증기를 거치기 전까지 힌트다.
- **구상 단계:** DNA·RNA 같은 생명과학 데이터로 넓히는 것은 아직 아이디어다.

카톡용 요약

그대로 복사해 보낼 수 있게 반말로 썼다.

> 연구개발은 아니고 버그 판정·검증 쪽이야
>
> key 인증이 되면 TCP 3단계 핸드셰이크랑 TLS로 연결하고 세션을 열어
>
> 그 안에서 코드 → 격자(보존·최대 원리 같은 증명된 경향) → 맥락 말소(연산자랑 위치값만) → e 사다리로 구간 → PINN이 구간 데이터 경향 학습(힌트) → 구간 산술은 코드가 → LLM이 새 코드 생성 → 검증기 통과한 것만 실행
>
> 결과는 API 서버 검사 거쳐서 DB에 저장하고, 프론트에서 시각화·페이징
>
> 인증 실패나 세션 만료면 바로 종료